

Stock Price Prediction: Time-Series Model

Andrew Pham - LSTM Model Implementation; Stephanie Snow - RNN Model Implementation; Ashley Andruss - GRU Model Implementation

Abstract

This research analyzed the accuracy and performance of three Recurrent Neural Network (RNN) architectures: Simple RNN, Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU). It used historical Apple stock data (1980-2020) to predict the next day's closing price. Recognizing the volatility of financial markets, 17 engineered features were derived beyond the standard OHLCV data to enhance pattern capture compared to using raw price data alone. The models were trained and evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R-Squared (R^2), and threshold accuracy metrics on a held-out test set after utilizing a separate validation set for model selection. The comparative analysis found that the GRU model delivered the strongest predictive performance, achieving the highest R^2 (0.983) and the lowest RMSE (\$7.55). The results also indicated that both gated architectures (LSTM and GRU) significantly outperformed the Simple RNN, underscoring the effectiveness of memory mechanisms for modeling this financial time series.

1 Introduction and Problem Description

1.1 Introduction

The stock market refers to a complex and dynamic trading network where people can buy and sell stock of public companies. The stock market plays a vital role when analysing economic health, facilitating the exchange of ownership to raise capital for growth and development, and is a powerful investment tool. Accurately predicting future stock prices is vital for a wide range of stakeholders, including traders, economists, and data scientists. Such predictions can significantly impact investment strategies, risk management, and policy making for both the short-term and long-term. The unpredictable nature of market volatility makes forecasting future prices a complex task. To analyze and adapt to these complexities, deep learning algorithms can be used to uncover patterns and trends from previous days.

There are five indicators commonly used to represent stock data over a period of time collectively known as OHLCV: Open, High, Low, Close, Volume. Open and Close refer to the first and last traded prices in a time frame. High and Low refer to the maximum and minimum price over a period of time, showing the range and volatility of prices. Lastly, Volume is the total number of shares traded during a period equaling the level of interest. Adjusted Volume is also included, which accounts for stock splits and provides a normalized view of trading activity over time.

1.2 Problem Description

This study investigates the prediction of the daily closing price for Apple Inc. (AAPL) stock, a task of significant financial relevance for investors, traders, and analysts seeking insights for decision-making. Forecasting AAPL prices accurately presents a considerable challenge due to the inherent volatility typical of tech stocks and the broader market, the significant noise present in financial time series data, and the influence of numerous complex and often unpredictable external factors, such as economic news, tech trends, and global events. Being that the stock market is open

from certain trading hours and days in various time zones, a lot can happen outside of those trading windows for worldly events. To tackle this challenge of volatility, this research utilizes historical daily data sourced via Kaggle (originally from Yahoo Finance [3]).

The dataset comprises nearly 10,000 entries from December 1980 to April 2020, including the standard OHLCV and Adjusted Close indicators. While the data represents trading days, gaps may exist between consecutive dates. Given the time-series nature of stock data, where past values influence future values, the primary goal is to predict the 'Close' price for the next time step based on historical patterns, making Recurrent Neural Networks (RNNs) a suitable modeling choice. To rigorously evaluate potential solutions, this study implements and compares three distinct RNN variants: a basic Simple RNN, a Long Short-Term Memory (LSTM) network, and a Gated Recurrent Unit (GRU) network. The comparison focuses on their predictive performance using a set of engineered features derived from the base data, assessed through metrics including Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R-Squared (R^2), and threshold accuracy.

2 Literature Review

Stock market prediction has been a challenging process for a long time due to non-linearity, volatility, and noise in financial data [1, 6]. Deep learning models have emerged in recent years and gained prominence for their ability to model time series data in financial forecasting, particularly RNNs and its variants LSTM and GRU. LSTM is one of the most accurate RNNs for financial forecasting because it uses input, forget, and output gates to forget some past stock data in order to predict future information [2, 5]. GRUs are another RNN variant that have been used recently in stock market prediction, but it has simpler architecture than LSTM with only two gates, the reset and update gates [2, 5]. GRU has been shown to have equal or slightly better performance when it comes to predicting stocks than the other models [1, 2, 5].

Overfitting can be an issue in deep learning models because it can lead to weak model generalization [2]. Beyond standard RNN, LSTM, and GRU, current research also shows that using different dimension reduction methods, such as LASSO and PCA, can offer a better prediction in stock prices and reduce overfitting when used with the LSTM and GRU models, especially with LASSO [2]. Common evaluation metrics used in research include Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE) [1, 5]. Hybrid models that combine neural networks with other techniques are also explored in stock prediction and are shown to outperform traditional neural networks along with LSTM [5, 6]. While RNNs can be useful in providing analysis into financial markets, there are still many economic, political, and socio-cultural factors that make accurately predicting markets difficult and in need of improvement [1, 5, 6].

3 Machine Learning Models, Methods, or Algorithms

This project employs three types of Recurrent Neural Networks (RNNs) to model and predict AAPL stock prices, allowing for a comparison of their architectures and performance on the same task. The models chosen are a Simple RNN (implemented by Stephanie Snow), a Long Short-Term Memory (LSTM) network (implemented by Andrew Pham), and a Gated Recurrent Unit (GRU) network (implemented by Ashley Andruss). The core methodology, including data

preprocessing, feature engineering, sequence creation, and evaluation metrics, was designed to be consistent across models, ensuring a fair comparison. Specific details are outlined below.

3.1 Preprocessing

Some of the original attributes, open, high, low, and close, are closely correlated to each other and thus transformed to avoid multicollinearity, as can be seen in Figure 1. For example, the difference between high and low is recorded as the range. The difference between close and open is recorded as the body (referring to a candlestick chart), and the body divided by the open is recorded as the percent change. To promote capturing previous data, the Simple Moving Average (SMA) and Exponential Moving Average (EMA) are included over a window. SMA is the average closing price in the window, while EMA is similar but emphasizes the more recent dates. Another feature included is the Rolling Standard Deviation (Rolling_SD) over windows which helps capture the volatility. Lastly, the expanded mean and standard deviation are included, which work similarly to the rolling variant, but the window size is from the start of the dataset to each entry. The window sizes for all these features are 5, 10, and 20-day windows. Since some of the entries aren't consecutive within 1 day, the time difference is included. In total, 17 features are used, with open and volume retained from the original set.

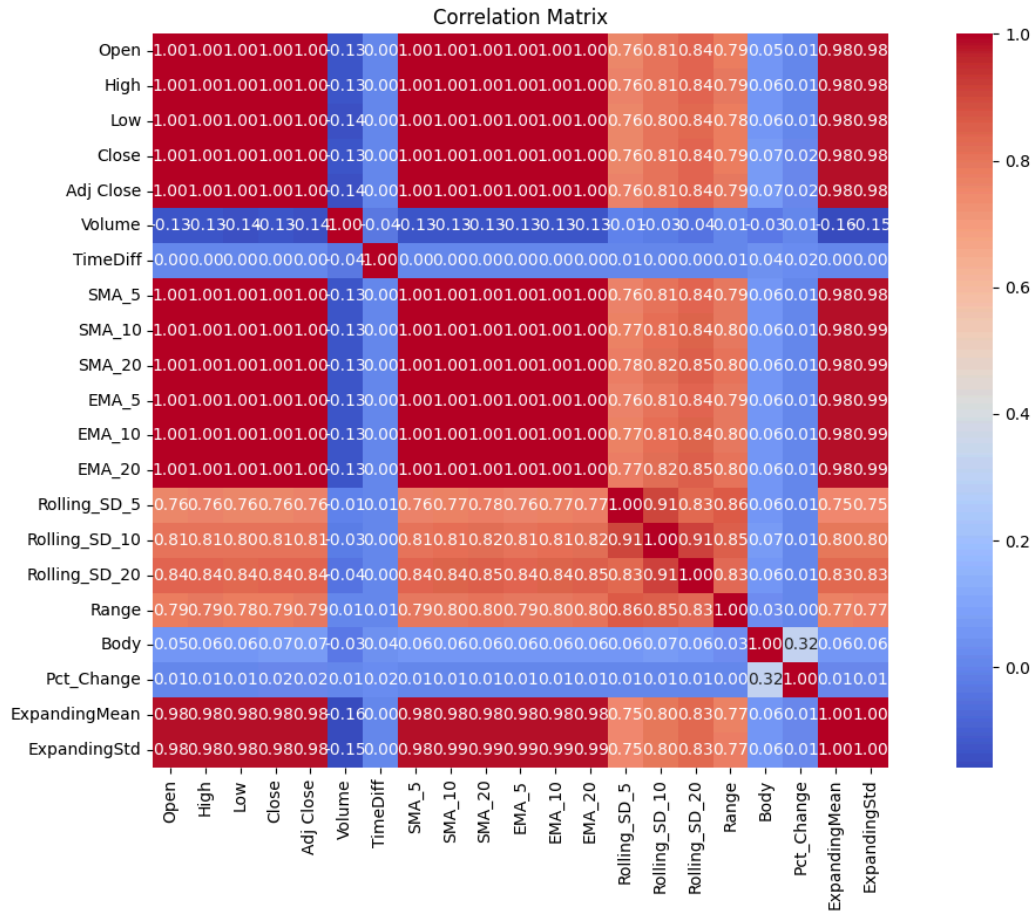


Figure 1: Correlation Matrix of All Engineered Features

The dataset is then split, with the first large percentage becoming the training set, while the remaining smaller percentage becomes the test/validation set; these generally are close to the

80/20 split. A MinMax scaler is fitted and applied to the training data, then applied to the testing data. Scaling by Min Max allows the model to converge faster and to treat the features more fairly.

3.2 Basic RNN

Basic RNN, also called Simple or Vanilla RNN, is the simplest form of Recurrent Neural Network. It processes sequential data by updating a hidden state at each time step in accordance with the previous hidden state and the current input. In this way, the RNN can learn dependencies between sequential data, allowing it to “remember” previous inputs.

The sequence length is assigned 40 entries (representing days in this case) which is an equal length of entries in a single batch to predict the next value. The model then stores the current input sequences and corresponding labels to train and test the model while making sure it does not go out of bounds. This format is the basis of the sequential input of the RNN as it helps the model learn temporal patterns based on previous data. The load and process data function loads the spreadsheet that contains the Apple stock information and runs the preprocessing code that engineers the features and extracts the target. It also originally splits the data into 80% training and 20% testing and uses MinMax scaling to normalize the features and labels to the range of [0, 1] for faster convergence and to help avoid gradient problems. The code will later split the training further into 72% training and 8% validation set in order to measure how well the model generalizes new data.

The RNN is then defined using the input size, hidden size, output size, number of layers, and dropout. The number of features used in this model is 17, which was determined during preprocessing. The hidden size is assigned 100 and the output size is 1 since it is predicting one price. The number of layers is left as its default 1 and dropout is set to 0.3 to help reduce overfitting. The model is designed to do a maximum of 500 epochs during training, however early stopping is implemented so that the model stops if there is no improvement on validation loss after the set 30 epochs of patience. By doing this, early stopping helps prevent overfitting. The learning rate for this model is 0.005 which determines how the weights in the RNN change to minimize the loss function. Finally, the Adam optimizer is used in training because it is a popular algorithm for deep learning models that can improve performance when used with neural networks.

3.3 Long Short-Term Memory (LSTM)

The input to the LSTM has a sequence length of 30 entries, which is approximately the last 30 days since the time difference mentioned in section 3.1. The model is made up of 1 LSTM layer with the input size equaling the number of features and 200 hidden nodes. After the LSTM layer, the output is passed into the dropout layer with a dropout rate of 20% to reduce overfitting. Finally, the output is passed into the fully connected layer, producing a single continuous value for the predicted closing price for the last timestep in the sequence.

The model is trained on the training set with a learning rate of 0.0005 to control the pace at which the model updates. The loss criterion used was the Mean Absolute Error (MAE) which is more robust to outliers compared to MSE. The Adam optimizer is employed for efficient gradient updates, and L2 regularization is added with a weight decay of 1e-4 to penalize large weights and further reduce overfitting. Additionally, a learning rate scheduler, ReduceLROnPlateau, monitors the validation loss and reduces the learning rate by 25% if the loss stagnates for 5 epochs, with a minimum allowable learning rate of 1e-5. The model is set to a maximum of 100 epochs, but it

will stop early when the validation loss doesn't improve for 15 epochs. To prevent the exploding gradient problem common in LSTMs, the gradients from backpropagation are clipped to a maximum norm of 10 before updating the model's weights for the next training step.

3.4 Gated Recurrent Unit (GRU)

Following the methodology applied to the LSTM model, the GRU network was made using the same set of 17 engineered input features derived for the historical AAPL data (including various moving averages, volatility measures, price range, percent change, and time difference indicators). Input sequences with a sequence length of 60 time steps, representing approximately 60 prior trading days, were fed into the model.

The GRU model architecture was designed for direct comparison and consists of a single GRU layer (`nn.GRU`) containing 200 hidden units. This layer processes the input sequences where each time step has 17 features. Immediately following the GRU layer, a dropout layer (`nn.Dropout`) with a probability of 0.2 is applied to the output of the final time step. This helps regularize the model and reduce the risk of overfitting. Finally, the output is passed through a fully connected linear layer (`nn.Linear`) that maps the 200-dimensional hidden state to a single continuous value, representing the predicted 'Close' price for the next time step.

The training configuration precisely mirrored that intended for the LSTM model to ensure comparability. The Mean Squared Error (MSE) was used as the loss function, suitable for this regression task. The Adam optimizer was employed with a learning rate initialized to 0.0005 and L2 regularization (weight decay) set to $1e-4$ (0.0001). A `ReduceLROnPlateau` learning rate scheduler monitored the validation loss, halving the learning rate if no improvement was observed for 5 epochs (down to a minimum of $1e-5$). Training was conducted using mini-batches of size 64 for a maximum of 500 epochs. To prevent exploding gradients, gradient clipping was applied with a maximum norm threshold of 10.0. For training purposes, the initial 80% training dataset was further partitioned by setting aside the final 10% of its generated sequences as a dedicated validation set used for learning rate scheduling and early stopping checks, ensuring the final 20% test set remained completely unseen. The full train, validation, and test split is described later. Early stopping halted the training process if the validation loss failed to improve for 10 consecutive epochs, thereby saving computation time and potentially selecting a better-generalized model state.

4 Experiment Results

This section details the results obtained from training and evaluating the Simple RNN, LSTM, and GRU models on the prepared AAPL dataset using the 17 engineered features. Key performance metrics and visualizations are presented for each model, followed by a comparative analysis.

4.1 Basic RNN

The Basic RNN model demonstrated improvement during the training by steadily decreasing in both the training loss and the validation loss using MSE Loss. Early stopping was triggered at epoch 383 where the best validation loss was 0.000218. This indicates that the model is generalizing well to the validation dataset. Also, it shows a low risk of overfitting because both the

training and validation loss decreased throughout the training even with the validation loss starting much higher than the training loss (Figure 2).

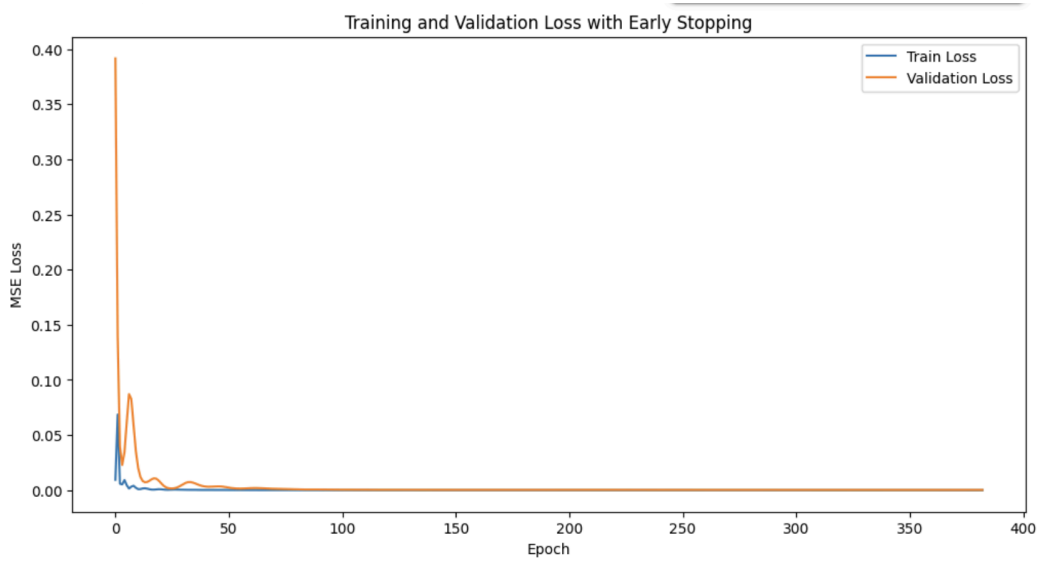


Figure 2: Basic RNN Model Training and Validation Loss Using MSE Loss vs. Epochs

Figure 3 displays the comparison between the actual closing price of the AAPL stock and the closing price of the stock as predicted based on the test data within a certain number of days.

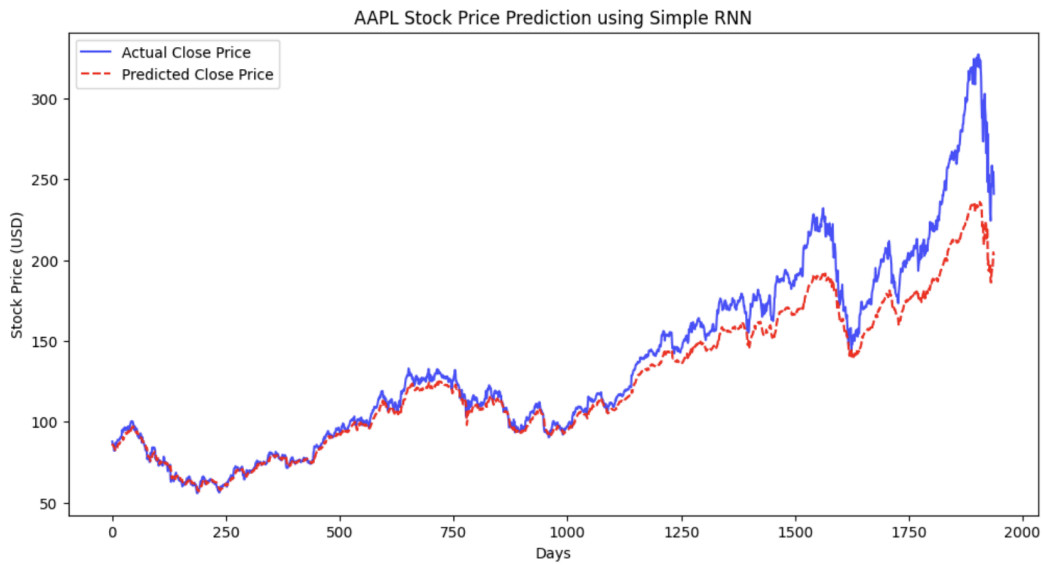


Figure 3: Actual vs. Predicted AAPL Close Prices Using the Basic RNN Model on the Test Set

The metric results of the test set are shown below.

Table 1: Basic RNN Performance

Parameter / Metric	Value
Model Configuration	
Data Split (Sequences)	72% Train / 8% Val / 20% Test

Sequence Length	40
Test Set Performance	
R²	0.8752
Mean Squared Error (MSE)	417.6166
Root Mean Squared Error (RMSE)	\$20.44
Mean Absolute Error (MAE)	\$12.13
Accuracy within 1%	11.40%
Accuracy within 2%	21.36%
Accuracy within 5%	51.19%
Accuracy within 10%	77.14%
Training Details	
Best Validation Loss (MSE)	0.000218
Epochs Ran	383
Training Time	419.87 seconds

The R² score suggests a high explained variance and better model fit because the dependent variable can be attributed based on the independent variables. MSE measures the average squared errors, which are the differences between the predicted and actual values. By squaring, MSE penalizes large errors, but in doing so it cannot be directly interpretable to US dollars. MAE measures the average of the absolute differences between predicted and actual values. Unlike MSE, it gives a more straightforward average error. Here, the MAE of 12.1338 means that the model's prediction is off by around \$12.13 a day. RMSE is the square root of MSE, which makes it translatable to US dollars. This means that the model's predictions deviate from the actual values on average around \$20.44 when focusing on larger errors.

The threshold accuracy showcases the model's prediction of the stock price as within 1% of the actual price 11.40% of the time, within 2% of the actual price 21.36% of the time, within 5% of the actual price 51.19% of the time, and within 10% of the actual price 77.14% of the time. To demonstrate the predictions of the Simple RNN model, it projected the closing price for the next trading day ending in 2020-04-01 to be **\$202.99**. While the model performs well in uncovering general trends and patterns, there is still room for improvement when it comes to price precision.

4.2 Long Short-Term Memory (LSTM)

As can be seen in Figure 4, the model is overall capturing the pattern as validation loss and is decreasing along with the training loss. However, it reaches a point where training will not go any lower and validation fluctuates around the same loss. While early stoppage was implemented, the model ran for the entire 100 epochs ending with a validation loss of 0.0002 taking 818.44 seconds. Validation loss being consistently smaller than the training loss could be due to the small size of 10%.

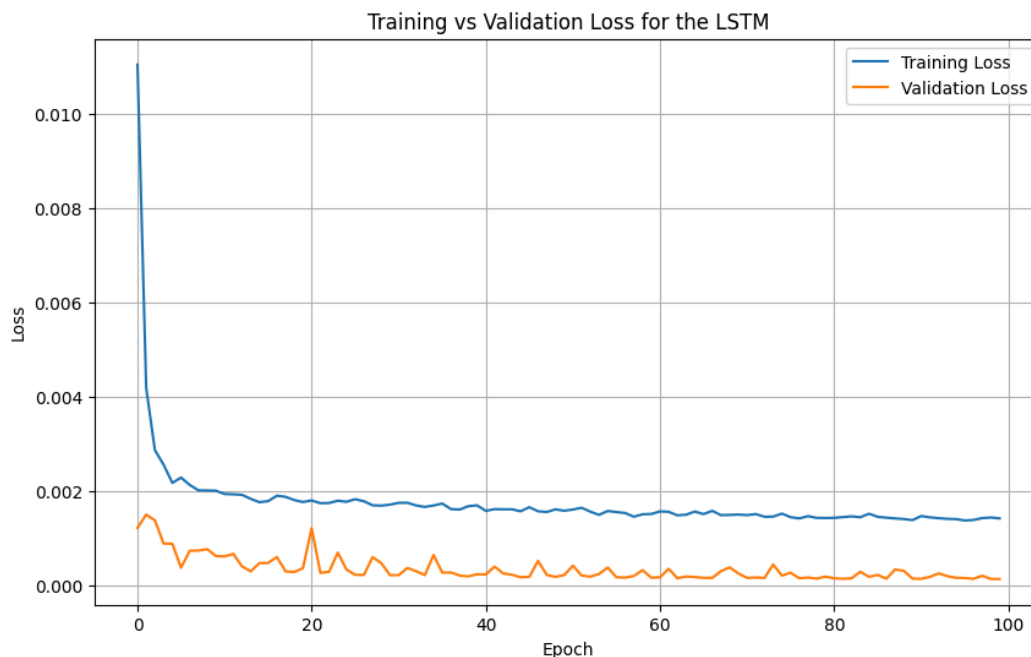


Figure 4: LSTM Model Training and Validation Loss Using MAE Loss vs. Epochs

Figure 5 illustrates the model's prediction accuracy compared to the actual closing prices. During the first 80% of the data, which was used for training, the predictions closely align with the real values. In the remaining unseen portion, although the accuracy decreases, the model still captures the overall trend.

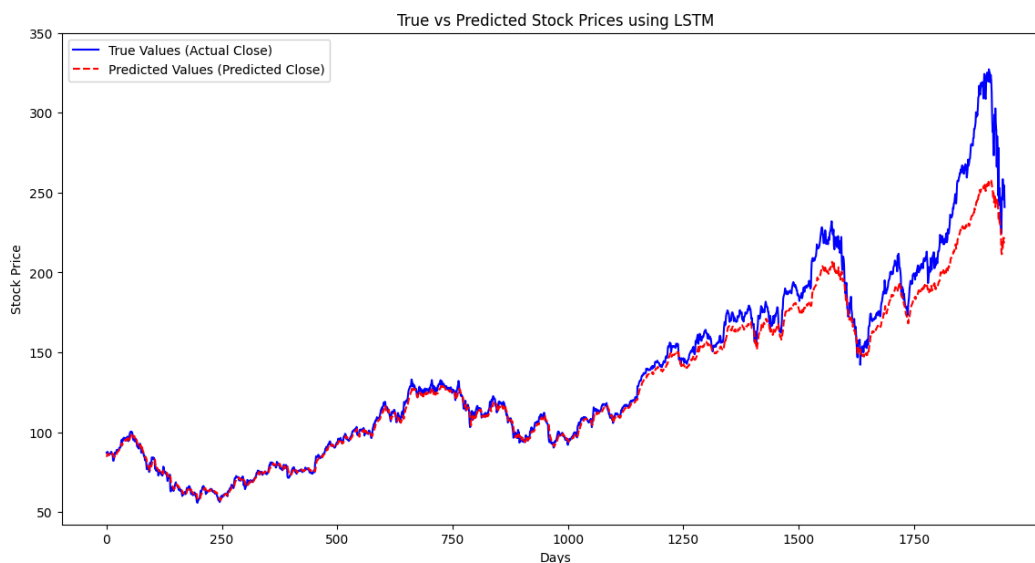


Figure 5: Actual vs. Predicted AAPL Close Prices Using the LSTM Model on the Test Set

Table 2: LSTM Performance

Parameter / Metric	Value
Model Configuration	
Data Split (Sequences)	70% Train / 10% Val / 20% Test

Sequence Length	30
Test Set Performance	
R²	0.9463
Mean Squared Error (MSE)	179.6243
Root Mean Squared Error (RMSE)	\$13.40
Mean Absolute Error (MAE)	\$7.08
Accuracy within 1%	22.23%
Accuracy within 2%	42.35%
Accuracy within 5%	74.59%
Accuracy within 10%	92.76%
Training Details	
Best Validation Loss (MAE)	0.0002
Epochs Ran	100
Training Time	818.44 seconds

The high R² value indicates the model explains 95% of the variance in the data. The relatively low MSE and RMSE values reflect that large prediction errors are uncommon, while the MAE shows that, on average, the model deviates only moderately from the actual price. In terms of accuracy, the model is less exact at more narrow thresholds but becomes significantly more dependable as the margin increases. For demonstration, applying the model to the data ending 2020-04-01 yielded a predicted closing price of **\$218.02** for the next trading day.

4.3 Gated Recurrent Unit (GRU)

The GRU model was trained using the 17 engineered features derived from the historical data. Relationships between these input features were initially assessed using the correlation matrix (Figure 1). The model's learning progress was tracked using Mean Squared Error (MSE) on both the training and the dedicated validation sets, as depicted over the training epochs in Figure 6.

The training stopped after 24 epochs due to the early stopping criterion being met, indicating that the validation loss had stopped improving. The entire training process was completed in approximately 298 seconds using CPU resources.

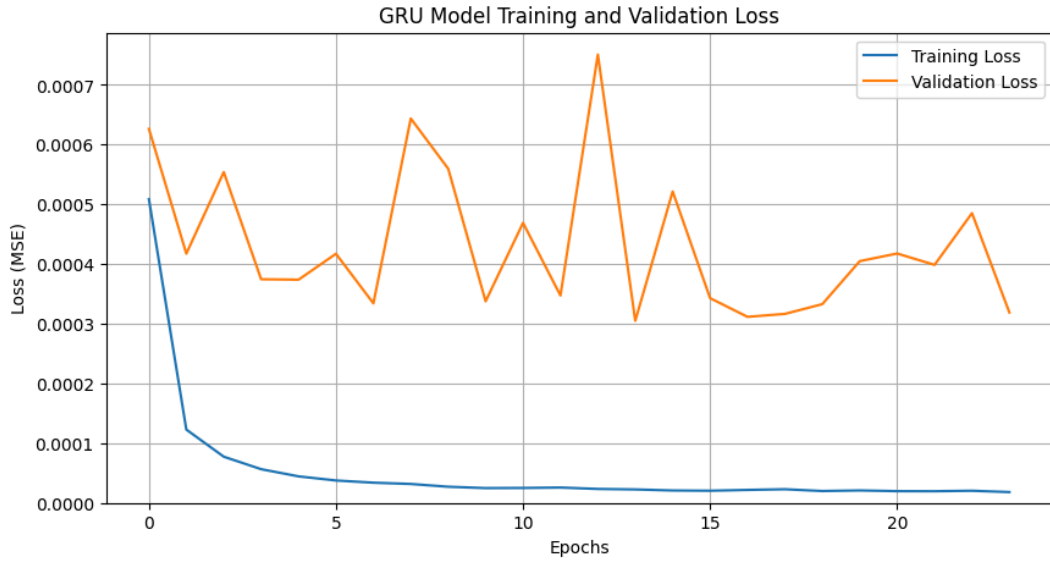


Figure 6: GRU Model Training and Validation Loss (MSE) vs. Epochs (Engineered Features)

In Figure 6, the trained GRU model's predictive capability was then evaluated on a completely unseen test set. In Figure 7, there is a visual comparison of the model's predictions against the actual closing prices for the test period. Quantitative model details and performance metrics calculated on the dataset are summarized in Table 3 below.

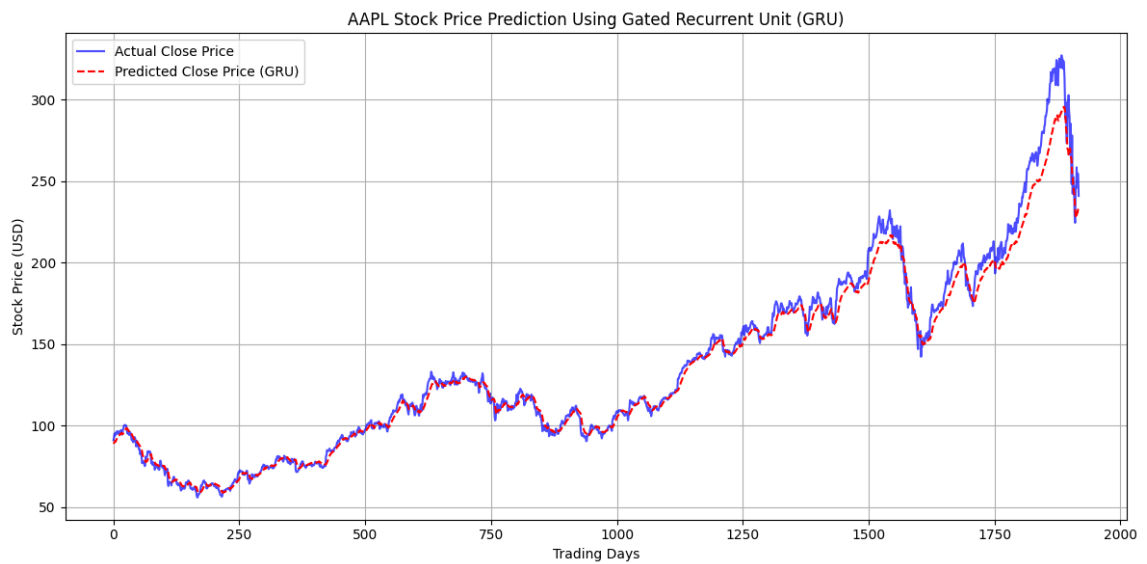


Figure 7: Actual vs. Predicted AAPL Close Prices using the Engineered GRU Model

Table 3: GRU Performance

Parameter / Metric	Value
Model Configuration	
Data Split (Sequences)	72% Train / 8% Val / 20% Test
Sequence Length	60
Test Set Performance	

R²	0.9830
Mean Squared Error (MSE)	56.9703
Root Mean Squared Error (RMSE)	\$7.55
Mean Absolute Error (MAE)	\$4.63
Accuracy within 1%	24.30%
Accuracy within 2%	44.16%
Accuracy within 5%	82.48%
Accuracy within 10%	98.64%
Training Details	
Best Validation Loss (MSE)	0.000305
Epochs Ran (Early Stopping)	24
Training Time	298.07 seconds

The R² value of 0.9830 indicates the model explains over 98% of the price variance in the test set. The Root Mean Squared Error (RMSE) of approximately \$7.55 provides an estimate of the typical prediction error magnitude, while the Mean Absolute Error (MAE) shows the average absolute error was about \$4.63. Threshold accuracy provides further insight into prediction closeness: while accuracy within the tightest 1-2% margins was modest (24-44%), 82.48% of the predictions fell within 5% of the actual closing price, and nearly all (98.64%) were within 10%. These results demonstrate the GRU model's strong capability to capture the stock's price movement trends using the engineered features, although precise daily point prediction remains inherently challenging. As a demonstration, applying the model to the data ending 2020-04-01 yielded a predicted closing price of **\$232.74** for the next trading day.

5 Conclusion

This study performed a comparative evaluation of Simple RNN, LSTM, and GRU architectures for predicting Apple's daily stock closing prices using historical data from 1980 to 2020. Recognizing the limitations of raw price data, an approach using 17 engineered features derived from OHLCV data was adopted across all models to capture market trends and volatility better. Each model was trained using an 80% data split, with a further 10% of the training sequences partitioned for validation to guide early stopping and learning rate adjustments, ensuring final evaluation occurred on a truly unseen 20% test set.

The final comparison across the three models revealed that the GRU architecture provided the best overall performance on the test set, achieving the highest R² (0.9830) and the lowest errors (RMSE \$7.55, MAE \$4.63). The LSTM model also demonstrated strong results (RMSE \$13.40), significantly outperforming the Simple RNN (RMSE \$20.44). The next day predicted close price for RNN (\$202.99), LSTM (\$218.02), and GRU (\$232.74) all vary based on the model architecture and performance. These findings highlight the advantage of gated RNN units (LSTM and GRU) over Simple RNNs for modeling complex financial time series, likely due to their enhanced ability to capture longer-term dependencies. The GRU, in this specific implementation with identical engineered features, proved slightly more effective than the LSTM. Key limitations of this study include the reliance solely on historical price and volume data, neglecting potentially influential external factors like market sentiment or macroeconomic news. The specific set of 17 engineered features, while beneficial, may not be optimal. Future research could explore

incorporating diverse data sources, investigating more advanced architectures like attention mechanisms or transformers, which have shown promise in sequence modeling, performing more extensive hyperparameter optimization, or focusing prediction on price returns rather than absolute levels to handle non-stationarity better.

6 Contributions

Each team member was responsible for implementing and evaluating a different type of RNN as part of our comparative analysis. Andrew implemented the LSTM model, Stephanie worked on the Basic RNN, and Ashley developed the GRU model. Each member handled data preprocessing, model training, and performance evaluation for their respective architecture. As for the report, the sections containing those models were written by the respective members. The other sections were done as followed:

Andrew was responsible for the Introduction and Problem Description (Section 1) and Preprocessing (Section 3.1). Ashley was responsible for initial NeurIPS formatting, Abstract, Introduction (Section 1.1), Machine Learning Models, Methods, or Algorithms (Section 3) statement, Experiment Results (Section 4) statement, and the Conclusion (Section 6). Stephanie was responsible for Literature Review (Section 2) and References (Section 7).

While we were each responsible for our own model section and corresponding code uploaded to GitHub, we also collaborated on peer reviewing and editing each other's work to ensure consistency and clarity through the report.

7 References

- [1] Chavhan, S., Raj, P., Raj, P., Dutta, A. K., & Rodrigues, J. J. P. C. (2024). Deep learning approaches for stock price prediction: A comparative study of LSTM, RNN, and GRU models. In *2024 9th International Conference on Smart and Sustainable Technologies (SpliTech)* (pp. 1–6). IEEE.
<https://doi.org/10.23919/SpliTech61897.2024.1061266>
- [2] Gao, Y., Wang, R., & Zhou, E. (2021). Stock prediction based on optimized LSTM and GRU models. *Scientific Programming*, 2021, 1–8. <https://doi.org/10.1155/2021/4055281>
- [3] Onyshchak, O. (2020). Stock Market Dataset [Data set]. Kaggle.
<https://doi.org/10.34740/KAGGLE/DSV/1054465>
- [4] Pham, A., Snow, S., & Andruss, A. (2025). Stock Market Prediction using RNN. *Github*.
<https://github.com/Andrew-QP/Stock-Market-Prediction-using-RNN>
- [5] Sako, K., Mpinda, B. N., & Rodrigues, P. C. (2022, May 7). Neural Networks for financial time series forecasting. *Entropy*, 24(5), 657. <https://doi.org/10.3390/e24050657>
- [6] Song, H., & Choi, H. (2023). Forecasting stock market indices using the recurrent neural network based hybrid models: CNN-LSTM, GRU-CNN, and ensemble models. *Applied Sciences*, 13(7), 4644.
<https://doi.org/10.3390/app130746>